

POLITECNICO DI TORINO



Lab 1

Exercise 3 - Filtering

Music and multimedia streaming over the Internet

Academic year 2025/2026

Full Name : MEHRAN BAYAT

Student Number : S320636.

Complete Report in Processing

Software used: Processing IDE 4

Language used: Processing Language

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026

©2025-26

README

Lab 1 Exercise 3: Filtering

Audio Analysis, Spectral Centroid Extraction, Mouse
Interaction, and Live Camera Tinting

Processing IDE 4, Sound Library, Video Library, FFT, and
Webcam Capture

Final_Report.md

Contents

1	Introduction	3
2	Statement of the Exercise	3
2.1	Audio Analysis	3
2.2	User Interaction	4
2.3	Video Camera	4
3	Objective of the Work	4
4	Theoretical Background	4
4.1	Fast Fourier Transform (FFT)	4
4.2	Spectral Centroid	5
4.3	Mapping to the Interval 0-255	5
4.4	Tinting the Camera Image	5
5	Software Environment and Libraries	6
5.1	Sound Library	6
5.2	Video Library	6
6	Development Procedure	6
6.1	Step 1: Creating the Sketch	7
6.2	Step 2: Installing the Required Libraries	7
6.3	Step 3: Preparing the Audio File	7
6.4	Step 4: Testing the Webcam Independently	8
6.5	Step 5: Testing the Audio and FFT Independently	8
6.6	Step 6: Combining All Components	8
7	Final Processing Code	9
8	Detailed Explanation of the Code	11
8.1	Imports	11
8.2	Global Variables	11
8.3	setup() Function	12
8.4	draw() Function	13
8.5	computeSpectralCentroid() Function	15
9	Validation of the Lab Requirements	15
9.1	Requirement 1: Audio file loaded and analyzed in real time using FFT	16
9.2	Requirement 2: Spectral centroid extracted from the frequency spectrum	16
9.3	Requirement 3: Spectral centroid mapped to 0-255	16
9.4	Requirement 4: Tracking mouseX and mouseY	16
9.5	Requirement 5: Mapping mouseX and mouseY to 0-255	16
9.6	Requirement 6: Capturing and displaying a live camera feed	17
9.7	Requirement 7: Using the three values as parameters for tint()	17
9.8	Requirement 8: Everything implemented in one single Processing project	17

10 Results and Observations	17
11 Discussion	18
12 Conclusion	18
13 References	19

1. Introduction

This report presents the complete development of **Exercise 3 - Filtering**, implemented in the Processing Language inside the Processing IDE. The goal of the exercise is to explore the relationship between sound, user interaction, and visual representation by creating a real-time audiovisual system. The system must combine audio analysis, mouse interaction, and a live webcam stream in a single Processing project. The final result must produce a live camera image whose color changes dynamically according to both the audio characteristics and the movements of the mouse.

The exercise is based on the idea that different kinds of input data can be translated into visual changes. In this case, the visual output is a webcam image, and its color is modified through the `tint()` function using three values:

1. the mapped value of the spectral centroid extracted from the audio signal,
2. the mapped value of `mouseX`,
3. the mapped value of `mouseY`.

The work was developed carefully and progressively. First, the required Processing libraries were installed. Then, the webcam was tested independently. After that, the audio playback and FFT analysis were tested independently. Finally, all components were merged into one final Processing sketch, as explicitly required by the lab.

2. Statement of the Exercise

According to the lab statement, Exercise 3 requires the implementation of a real-time interactive multimedia system in Processing. The requested components are the following.

2.1 Audio Analysis

The exercise requires:

1. loading an audio file named `audio.wav`,
2. analyzing it in real time using FFT (Fast Fourier Transform),
3. extracting the spectral centroid from the frequency spectrum,
4. mapping the centroid value to the interval $0 - 255$.

2.2 User Interaction

The exercise requires:

1. tracking `mouseX` and `mouseY` in real time,
2. mapping both coordinates to values between 0 and 255.

2.3 Video Camera

The exercise requires:

1. capturing and displaying a live camera feed,
2. using the three values - mapped spectral centroid, mapped `mouseX`, and mapped `mouseY` - as parameters of the `tint()` function.

The expected result is therefore a camera image whose color changes dynamically according to sound properties and user input.

3. Objective of the Work

The objective of this work is to create a system in which:

- the audio signal influences the visual output through the spectral centroid,
- the user influences the visual output through the mouse position,
- the camera image becomes the final visual medium on which those changes are applied.

This makes the program an interactive audiovisual interface. The system does not simply play sound or show the webcam feed independently. Instead, it establishes a direct connection between what is heard, how the user moves the mouse, and how the camera image appears on the screen.

4. Theoretical Background

4.1 Fast Fourier Transform (FFT)

The Fast Fourier Transform is a method used to analyze a signal in the frequency domain. Instead of representing the audio only as a waveform in time, FFT decom-

poses the signal into frequency components. Each component corresponds to a specific frequency band and has a magnitude.

In this exercise, the FFT is applied in real time to the audio file. At each frame of the Processing sketch, the frequency spectrum is computed and stored in an array. This array contains the magnitudes of the different frequency bins.

4.2 Spectral Centroid

The spectral centroid is the main audio descriptor used in this exercise. It represents the center of mass of the frequency spectrum and is often associated with the perceptual brightness of the sound. The centroid is defined by the formula:

$$C = \frac{\sum_{k=0}^{N-1} f_k \cdot X(k)}{\sum_{k=0}^{N-1} X(k)} \quad (1)$$

where:

- f_k is the frequency of the k -th FFT bin,
- $X(k)$ is the magnitude of the k -th FFT bin,
- N is the total number of FFT bins.

This formula gives a weighted average of the frequencies present in the signal. Higher values of the spectral centroid indicate that the sound contains more energy at higher frequencies, while lower values indicate that the energy is concentrated more in the low-frequency region.

4.3 Mapping to the Interval 0-255

The exercise explicitly requires mapping the spectral centroid and the mouse coordinates to the range 0 – 255. This interval is important because it is the standard RGB range used by Processing for color representation. The function `map()` is used for this purpose.

For example, if the spectral centroid is measured in Hertz, it must be converted into the interval 0 – 255 before it can be used as a tint parameter. Similarly, `mouseX` and `mouseY` must be converted from screen coordinates to values inside the same interval.

4.4 Tinting the Camera Image

The function `tint()` in Processing applies a color filter to images before they are drawn on the screen. In this exercise, the live webcam image is displayed with a tint such that:

- the red component comes from the mapped spectral centroid,
- the green component comes from the mapped `mouseX`,
- the blue component comes from the mapped `mouseY`.

This means that the appearance of the webcam image changes continuously and dynamically as the sound evolves and as the user moves the mouse.

5. Software Environment and Libraries

The implementation was carried out in:

- Processing IDE 4
- Processing Language

Two official Processing libraries were required.

5.1 Sound Library

The Sound library was used to:

- load the audio file,
- play the audio in real time,
- perform FFT analysis,
- access the sampling rate of the sound engine.

5.2 Video Library

The Video library was used to:

- detect the connected camera,
- capture live frames,
- display the webcam feed in the Processing window.

6. Development Procedure

The implementation was completed in a step-by-step manner so that each subsystem could be verified independently before being integrated into the final project.

6.1 Step 1: Creating the Sketch

A new Processing sketch was created and saved under the name:

```
Exercise3Filtering
```

The project had to remain a single Processing project, in full accordance with the lab requirement.

6.2 Step 2: Installing the Required Libraries

The following libraries were installed using:

```
Sketch -> Import Library -> Manage Libraries
```

The installed libraries were:

- Sound by The Processing Foundation
- Video Library for Processing 4 by The Processing Foundation

After installation, the following imports were added at the top of the sketch:

Listing 1: Required library imports

```
1 import processing.sound.*;  
2 import processing.video.*;
```

6.3 Step 3: Preparing the Audio File

The audio file required by the exercise was placed inside the sketch data folder so that the Processing program could load it correctly. The sketch was therefore organized conceptually as follows:

Listing 2: Sketch structure

```
Exercise3Filtering/  
Exercise3Filtering.pde  
data/  
audio.wav
```

6.4 Step 4: Testing the Webcam Independently

Before combining everything, the webcam was tested alone to ensure that the Video library and camera access were working correctly. A minimal sketch was written to:

- detect connected cameras,
- initialize the first available webcam,
- read frames in real time,
- display the webcam image in the window.

This test succeeded, which confirmed that the camera subsystem worked properly.

6.5 Step 5: Testing the Audio and FFT Independently

After that, the audio analysis was tested independently. A sketch was written to:

- load `audio.wav`,
- loop the sound continuously,
- connect the sound file to the FFT analyzer,
- compute the spectrum in each frame,
- compute the spectral centroid from that spectrum,
- map the centroid to 0 – 255,
- display both the spectral centroid in Hz and its mapped value.

This test also succeeded. Therefore, the audio subsystem and spectral centroid computation were validated before the final integration.

6.6 Step 6: Combining All Components

After both subsystems had been tested independently, they were merged into one final sketch. In the final implementation:

- the audio file is played and analyzed,
- the spectral centroid is computed and mapped,
- the webcam feed is captured and displayed,
- `mouseX` and `mouseY` are tracked and mapped,
- the webcam image is tinted using the three mapped values.

7. Final Processing Code

The final code that satisfies all the requirements of the exercise is reported below.

Listing 3: Complete Processing implementation of Exercise 3

```
1  import processing.sound.*;
2  import processing.video.*;
3
4  Sound engine;
5  SoundFile audio;
6  FFT fft;
7  Capture cam;
8
9  int bands = 512;
10 float[] spectrum = new float[bands];
11
12 float sampleRate = 44100.0;
13 float centroidHz = 0;
14 float centroidMapped = 0;
15 float mouseMappedX = 0;
16 float mouseMappedY = 0;
17
18 void setup() {
19     size(640, 480);
20
21     // ---- AUDIO SETUP ----
22     engine = new Sound(this);
23     sampleRate = Sound.sampleRate();
24
25     audio = new SoundFile(this, "audio.wav");
26     audio.loop();
27
28     fft = new FFT(this, bands);
29     fft.input(audio);
30
31     // ---- CAMERA SETUP ----
32     String[] cameras = Capture.list();
33
34     if (cameras == null || cameras.length == 0) {
35         println("No camera found.");
36         exit();
37         return;
38     }
39
40     cam = new Capture(this, cameras[0]);
41     cam.start();
42
43     textSize(16);
44 }
45
46 void draw() {
47     background(0);
```

```
48
49 // Read webcam frame
50 if (cam.available()) {
51     cam.read();
52 }
53
54 // Analyze sound spectrum
55 fft.analyze(spectrum);
56
57 // Compute spectral centroid
58 centroidHz = computeSpectralCentroid(spectrum);
59
60 // Map centroid to 0..255
61 float nyquist = sampleRate / 2.0;
62 centroidMapped = map(centroidHz, 0, nyquist, 0, 255);
63 centroidMapped = constrain(centroidMapped, 0, 255);
64
65 // Map mouse values to 0..255
66 mouseMappedX = map(mouseX, 0, width, 0, 255);
67 mouseMappedX = constrain(mouseMappedX, 0, 255);
68
69 mouseMappedY = map(mouseY, 0, height, 0, 255);
70 mouseMappedY = constrain(mouseMappedY, 0, 255);
71
72 // Apply tint
73 tint(centroidMapped, mouseMappedX, mouseMappedY);
74 image(cam, 0, 0, width, height);
75 noTint();
76
77 // Information panel
78 fill(255, 230);
79 rect(10, 10, 270, 100);
80
81 fill(0);
82 text("Centroid (Hz): " + nf(centroidHz, 0, 2), 20, 35);
83 text("Centroid mapped: " + nf(centroidMapped, 0, 2), 20, 55);
84 text("Mouse X mapped: " + nf(mouseMappedX, 0, 2), 20, 75);
85 text("Mouse Y mapped: " + nf(mouseMappedY, 0, 2), 20, 95);
86 }
87
88 float computeSpectralCentroid(float[] spectrumData) {
89     float weightedSum = 0;
90     float magnitudeSum = 0;
91
92     for (int k = 0; k < spectrumData.length; k++) {
93         float frequency = k * sampleRate / (2.0 * spectrumData.length);
94         float magnitude = spectrumData[k];
95
96         weightedSum += frequency * magnitude;
97         magnitudeSum += magnitude;
98     }
99
100     if (magnitudeSum == 0) {
101         return 0;
```

```
102 }  
103  
104 return weightedSum / magnitudeSum;  
105 }
```

8. Detailed Explanation of the Code

8.1 Imports

The lines:

```
1 import processing.sound.*;  
2 import processing.video.*;
```

import the Sound and Video libraries. These are necessary to use the classes `SoundFile`, `FFT`, and `Capture`.

8.2 Global Variables

The code declares four main objects:

- `Sound engine`; for initializing the sound system,
- `SoundFile audio`; for loading and playing the audio file,
- `FFT fft`; for performing FFT analysis,
- `Capture cam`; for reading frames from the webcam.

The number of FFT bands is set to:

```
1 int bands = 512;
```

A spectrum array of the same length is created:

```
1 float[] spectrum = new float[bands];
```

This array stores the magnitude of each FFT frequency bin.

Several floating-point variables are also declared:

- `sampleRate` stores the sound engine sampling rate,
- `centroidHz` stores the spectral centroid in Hertz,
- `centroidMapped` stores the centroid mapped to 0 – 255,

- `mouseMappedX` stores the mapped horizontal mouse position,
- `mouseMappedY` stores the mapped vertical mouse position.

8.3 `setup()` Function

The `setup()` function initializes everything only once when the sketch starts.

Window Creation

The line:

```
1 size(640, 480);
```

creates the main window.

Audio Initialization

The lines:

```
1 engine = new Sound(this);  
2 sampleRate = Sound.sampleRate();
```

initialize the sound engine and retrieve its sample rate.

Then the audio file is loaded and looped:

```
1 audio = new SoundFile(this, "audio.wav");  
2 audio.loop();
```

This means that the audio is played continuously.

Next, the FFT analyzer is created and connected to the audio stream:

```
1 fft = new FFT(this, bands);  
2 fft.input(audio);
```

This ensures that every call to `fft.analyze(spectrum)` will analyze the currently playing audio file.

Camera Initialization

The code detects all available cameras:

```
1 String[] cameras = Capture.list();
```

If no camera is found, the program exits safely:

```
1 if (cameras == null || cameras.length == 0) {
2     println("No camera found.");
3     exit();
4     return;
5 }
```

If at least one camera exists, the first camera is selected and started:

```
1 cam = new Capture(this, cameras[0]);
2 cam.start();
```

8.4 draw() Function

The `draw()` function runs continuously and manages all real-time operations.

Reading the Webcam Frame

The following code checks if a new camera frame is available and then reads it:

```
1 if (cam.available()) {
2     cam.read();
3 }
```

FFT Analysis

The current audio spectrum is computed with:

```
1 fft.analyze(spectrum);
```

After this call, the array `spectrum` contains the spectral magnitudes of the current audio frame.

Spectral Centroid Computation

The spectral centroid is computed with:

```
1 centroidHz = computeSpectralCentroid(spectrum);
```

The auxiliary function `computeSpectralCentroid()` implements the formula given in the exercise statement.

Mapping the Spectral Centroid

The spectral centroid is originally measured in Hertz. It is converted into the RGB interval 0 – 255 by:

```
1 float nyquist = sampleRate / 2.0;
2 centroidMapped = map(centroidHz, 0, nyquist, 0, 255);
3 centroidMapped = constrain(centroidMapped, 0, 255);
```

The use of the Nyquist frequency $\frac{\text{sampleRate}}{2}$ is correct because the FFT covers frequencies from 0 Hz to half the sampling rate.

Mapping Mouse Coordinates

The horizontal and vertical mouse positions are mapped into the same color range:

```
1 mouseMappedX = map(mouseX, 0, width, 0, 255);
2 mouseMappedX = constrain(mouseMappedX, 0, 255);
3
4 mouseMappedY = map(mouseY, 0, height, 0, 255);
5 mouseMappedY = constrain(mouseMappedY, 0, 255);
```

These two lines satisfy the user interaction requirement of the exercise.

Applying the Tint

The camera image is tinted with the three mapped values:

```
1 tint(centroidMapped, mouseMappedX, mouseMappedY);
2 image(cam, 0, 0, width, height);
3 noTint();
```

This means:

- the red channel depends on the spectral centroid,
- the green channel depends on the horizontal mouse position,
- the blue channel depends on the vertical mouse position.

The webcam image is therefore directly controlled by both sound properties and user input.

Displaying Numeric Information

An information panel is drawn on top of the webcam image so that the values can be observed in real time:

```

1  fill(255, 230);
2  rect(10, 10, 270, 100);
3
4  fill(0);
5  text("Centroid (Hz): " + nf(centroidHz, 0, 2), 20, 35);
6  text("Centroid mapped: " + nf(centroidMapped, 0, 2), 20, 55);
7  text("Mouse X mapped: " + nf(mouseMappedX, 0, 2), 20, 75);
8  text("Mouse Y mapped: " + nf(mouseMappedY, 0, 2), 20, 95);

```

These on-screen values are useful for understanding how the system behaves while it is running.

8.5 computeSpectralCentroid() Function

The function:

```

1  float computeSpectralCentroid(float[] spectrumData)

```

computes the spectral centroid exactly as required by the exercise.

For each FFT bin:

- the corresponding frequency f_k is computed,
- the magnitude $X(k)$ is taken from the spectrum array,
- the weighted sum $f_k \cdot X(k)$ is accumulated,
- the total magnitude is accumulated.

Finally, the function returns:

$$\frac{\sum_{k=0}^{N-1} f_k X(k)}{\sum_{k=0}^{N-1} X(k)} \quad (2)$$

If the denominator is zero, the function returns zero to avoid division by zero.

9. Validation of the Lab Requirements

This section verifies point by point that the implementation satisfies all the requirements of the lab.

9.1 Requirement 1: Audio file loaded and analyzed in real time using FFT

This requirement is satisfied by:

```
1 audio = new SoundFile(this, "audio.wav");
2 audio.loop();
3
4 fft = new FFT(this, bands);
5 fft.input(audio);
6 fft.analyze(spectrum);
```

The audio file is loaded, played continuously, and analyzed in real time.

9.2 Requirement 2: Spectral centroid extracted from the frequency spectrum

This requirement is satisfied by the function:

```
1 centroidHz = computeSpectralCentroid(spectrum);
```

The function computes the spectral centroid from the FFT magnitudes and the corresponding frequency of each bin.

9.3 Requirement 3: Spectral centroid mapped to 0-255

This requirement is satisfied by:

```
1 float nyquist = sampleRate / 2.0;
2 centroidMapped = map(centroidHz, 0, nyquist, 0, 255);
3 centroidMapped = constrain(centroidMapped, 0, 255);
```

9.4 Requirement 4: Tracking mouseX and mouseY

This requirement is satisfied because mouseX and mouseY are read in every frame inside draw().

9.5 Requirement 5: Mapping mouseX and mouseY to 0-255

This requirement is satisfied by:

```
1 mouseMappedX = map(mouseX, 0, width, 0, 255);
2 mouseMappedX = constrain(mouseMappedX, 0, 255);
```

```
3  
4 mouseMappedY = map(mouseY, 0, height, 0, 255);  
5 mouseMappedY = constrain(mouseMappedY, 0, 255);
```

9.6 Requirement 6: Capturing and displaying a live camera feed

This requirement is satisfied by:

```
1 String[] cameras = Capture.list();  
2 cam = new Capture(this, cameras[0]);  
3 cam.start();  
4  
5 if (cam.available()) {  
6     cam.read();  
7 }  
8 image(cam, 0, 0, width, height);
```

9.7 Requirement 7: Using the three values as parameters for tint()

This requirement is satisfied by:

```
1 tint(centroidMapped, mouseMappedX, mouseMappedY);
```

9.8 Requirement 8: Everything implemented in one single Processing project

This requirement is satisfied because the complete solution is contained in one Processing sketch.

10. Results and Observations

When the final sketch runs, the webcam feed appears live inside the Processing window. At the same time, the audio file is played continuously in the background and analyzed frame by frame using FFT.

The observed behavior of the final system is the following:

- The red channel of the webcam image changes according to the spectral centroid.

If the audio contains more high-frequency energy, the spectral centroid increases and the red component becomes stronger.

- The green channel changes as the user moves the mouse horizontally. Moving the mouse from left to right increases the value of `mouseMappedX`.
- The blue channel changes as the user moves the mouse vertically. Moving the mouse from top to bottom changes the value of `mouseMappedY`.

Because all three values are applied simultaneously, the image color is continuously and dynamically modified by both the sound and the user input. This produces a direct interaction between listening, movement, and vision.

During testing, the audio-only version showed numerical values for the spectral centroid and its mapped equivalent, which confirmed that the FFT and centroid computation were functioning properly before the final integration. After integration, the final system behaved as intended, with a visible tinted webcam feed and real-time changes caused by both sound and mouse movement.

11. Discussion

This exercise demonstrates how different forms of media data can be integrated inside a single interactive system. The audio file is not only played but also analyzed in real time. The extracted descriptor, namely the spectral centroid, becomes a control parameter for a visual transformation. At the same time, user input through the mouse introduces a second and third control source. Finally, the webcam feed provides a live visual signal that reacts to those controls.

The result is meaningful from both a technical and conceptual perspective. Technically, it shows how to use the Processing ecosystem to work with audio analysis, real-time interaction, and live video capture. Conceptually, it illustrates how a sound characteristic and human gestures can be translated into visual changes, producing an audiovisual environment rather than a simple static display.

The use of the spectral centroid is particularly appropriate in this context because it is directly related to the distribution of spectral energy, making it a useful descriptor for controlling brightness-like visual properties. The use of `mouseX` and `mouseY` adds an intuitive interactive layer, allowing the user to shape the image actively while the sound is simultaneously influencing it.

12. Conclusion

This work successfully completed all the requirements of Exercise 3. A single Processing project was created that:

1. loads an audio file,
2. analyzes it in real time using FFT,
3. computes the spectral centroid,
4. maps the centroid to the interval 0 – 255,
5. tracks `mouseX` and `mouseY`,
6. maps both mouse coordinates to 0 – 255,
7. captures and displays a live webcam feed,
8. applies `tint()` to the webcam image using the three mapped values.

The final result is a dynamic audiovisual Processing sketch in which the visual appearance of the webcam feed is continuously shaped by sound properties and user interaction. The exercise therefore fulfills its intended purpose of exploring the relationship between sound, interaction, and visual representation through a real-time multimedia system.

13. References

1. Processing. Processing Language Reference. Available at: <https://processing.org/reference/>
2. Processing. Processing Libraries. Available at: <https://processing.org/reference/libraries/>
3. Processing. Sound Library Reference. Available at: <https://processing.org/reference/libraries/sound/>
4. Processing. Video Library Reference. Available at: <https://processing.org/reference/libraries/video/>
5. Processing. `map()` Reference. Available at: https://processing.org/reference/map_
6. Processing. `tint()` Reference. Available at: https://processing.org/reference/tint_
7. Processing. Sound Tutorial. Available at: <https://processing.org/tutorials/sound/>